# ARE SELF-EVOLVING AGENT SKILLS LEARNING SKILLS?

*Anonymous Submission*


## ABSTRACT

Self-evolving agent skills turn execution experience into persistent instructions and code, but improved performance on the source task leaves open what has been learned. We study this question through repeated standard-task evaluations, controlled task variations, and inspection of the resulting skills and their use. Our diagnosis distinguishes source-task utility from usefulness under changes to input representation, task instances, and requirements. We examine SkillEvolver, SPARK, and CoEvoSkills, connecting their observed reuse patterns to the decision rules preserved in their skills and the feedback used during skill construction. The results reveal different scopes of generalization despite positive source-task gains, and motivate evaluating skills by how they guide decisions when the task changes.


***Index Terms*—** Agent skills, self-evolution, diagnostic evaluation, skill reuse, generalization

## 1. INTRODUCTION

Persistent agent skills store instructions, scripts, and references that an executor can reuse across tasks. Skill-generation and evolution methods turn execution experience into these external artifacts without changing the executor's model parameters. SkillEvolver refines skills from their deployment outcomes, SPARK distills environment-verified trajectories, and CoEvoSkills jointly evolves skills and a surrogate verifier [1–3]. Their common goal is to make experience useful beyond the attempt that produced it.

Success on the source task does not reveal the scope of that usefulness. A skill can help by encoding a procedure that adapts to new inputs, or by preserving choices that happen to work on one instance. Both can raise a benchmark score. The distinction becomes visible when the executor must reinterpret an input, solve a replacement instance, or satisfy an additional requirement while using the same skill. These changes test what the artifact preserves from experience and what the executor must still infer.

We investigate this distinction as a diagnostic study of skill reuse. We first measure each method's gain over a same-task no-skill baseline in two standard-task runs. We then freeze the skills and examine their utility under changes to representation, instances, and requirements. Finally, we inspect contrasting examples to connect successful and unsuccessful reuse to the guidance expressed in the skills. This sequence moves from whether a skill helps to where it helps and what its contents suggest about the difference.

The resulting picture differs across methods. All three show positive paired gains in both standard-task runs. Under task variation, SkillEvolver retains positive average gains across all three change types, SPARK's gains depend on the type of change, and CoEvoSkills' average gains reverse sign. Instance changes produce the clearest separation. Thus recurring source-task gains and generalization describe different properties of an evolved skill.

The case evidence makes this distinction concrete. In signal detection, selecting a strong candidate is insufficient when the signal can be confused with background variability or a harmonic. In scheduling, a feasible output can still violate the task's repair rules. Successful reuse in these examples accompanies skills that preserve criteria for choosing and checking a result. We interpret these differences in light of the methods' feedback and skill-construction procedures.

This paper contributes a diagnosis of skill learning through three complementary forms of evidence:

- **Repeated utility measurement.** Six standard-task runs establish the observed paired benefits of SkillEvolver, SPARK, and CoEvoSkills on their evaluated tasks.
- **Task-variation analysis.** Representation, instance, and requirement changes expose different scopes of frozen-skill usefulness across the recorded variant evaluations.
- **Skill-level interpretation.** Contrasting reuse examples connect outcomes to preserved decision criteria and motivate an explanation based on feedback coverage and abstraction from the source instance.

## 2. PROBLEM FORMULATION

### 2.1. Skill Revision and Evaluation

At revision round $r$, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where $s_r$ is the current skill, $\tau_r$ is an execution trace, and $f_r$ is feedback from execution and evaluation. The operator $O$ denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not $s_{r+1}$.

The agent then executes task $x$ with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where $z_r$ represents model sampling, tool use, and environment state, and $o_r$ is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

# 自进化智能体技能是否在学习技能?

匿名提交


## 摘要

自进化代理技能将执行经验转化为持久指令和代码，但源任务性能的提升仍留下了已学习到什么的问题。我们通过重复标准任务评估、控制任务变化以及检查生成的技能及其使用情况来研究这一问题。我们的诊断区分了源任务效用与输入表示、任务实例和需求变化时的有用性。我们考察了SkillEvolver、SPARK和CoEvoSkills，将它们观察到的重用模式与其技能中保留的决策规则以及技能构建过程中使用的反馈联系起来。结果表明，尽管源任务收益为正，但泛化范围不同，并促使我们通过评估技能在任务变化时如何指导决策来评估技能。


索引词— 代理技能，自进化，诊断评估，技能重用，泛化

## 1. 引言

持久的代理技能存储指令、脚本和参考，执行者可以在不同任务中重复使用。技能生成和进化方法将执行经验转化为这些外部工件，而不会改变执行者的模型参数。SkillEvolver 从部署结果中优化技能，SPARK 提炼经过环境验证的轨迹，CoEvoSkills 联合进化技能和一个代理验证器 [1–3]。它们的共同目标是使经验在产生它的尝试之外仍然有用。

在源任务上的成功并不能揭示其有用性的范围。一个技能可以通过编码一个适应新输入的流程来提供帮助，或者通过保留在某个实例上偶然有效的选择来提供帮助。两者都可以提高基准分数。当执行者必须在使用相同技能时重新解释输入、解决替换实例或满足附加要求时，这种区别才会变得明显。这些变化测试了工件从经验中保留了什么，以及执行者仍然需要推断什么。

我们作为技能重用的诊断研究来调查这种区别。我们首先在两个标准任务运行中测量每种方法相对于同一任务无技能基线的收益。然后冻结技能，并在表示、实例和需求发生变化的情况下检查它们的效用。最后，我们检查对比示例，以将成功和失败的重复与技能中表达的指导联系起来。这一系列从技能是否有助于开始，到它在哪里有助于以及它的内容暗示了什么差异。

三种方法呈现出不同的结果。它们在标准任务的两次运行中均有正向配对增益。在任务变体上，SkillEvolver 在全部三类变化下均保持正向平均增益；SPARK 的收益取决于变化类型，CoEvoSkills 则由正转负。实例变化的分化最明显。因此，源任务收益反复出现与泛化是技能的不同属性。

案例证据使这种区分变得具体。在信号检测中，当信号可能与背景变化性或谐波混淆时，选择一个强候选者是不够的。在调度中，一个可行的输出仍然可能违反任务的修复规则。在这些例子中成功的重用伴随着能够保存选择和检查结果标准的技能。我们从这些方法的反馈和技能构建程序的角度来解释这些差异。

本文通过三种互补的证据形式对技能学习进行诊断：

- **重复效用测量**。 六次标准任务运行确立了SkillEvolver、SPARK和CoEvoSkills在其评估任务上观察到的配对收益。
- **任务变化分析**。 表示、实例和需求变化揭示了在不同记录的变体评估中冻结技能有用性的不同范围。
- **技能级别解释**。 对比重用示例将结果与保存的决策标准联系起来，并基于反馈覆盖和从源实例中抽象的解释进行说明。

## 2. 问题定义

### 2.1. 技能修订与评估

在修订轮次 $r$ 中，优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 $s_r$ 是当前技能，$\tau_r$ 是执行轨迹，而 $f_r$ 是来自执行和评估的反馈。运算符 $O$ 表示方法完整的更新路径，包括其原生合并、验证、选择或采纳机制。一个被拒绝或从未被采纳的生成候选者不是 $s_{r+1}$。

随后，智能体使用冻结技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

$z_r$ 表示模型采样、工具使用和环境状态，而 $o_r$ 是产生的输出工件或状态变化。验证器将输出映射到分数：

$$y_r = V(x, o_r). \tag{3}$$

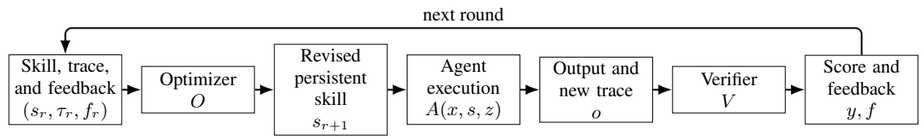



**Fig. 1**. Skill evolution and deployment. Our evaluations examine the utility of the completed skill and its reuse under task changes; skill contents and execution traces provide evidence for interpreting the observed behavior.

### 2.2. Diagnosing What a Skill Has Learned

Let $s_x$ denote the completed skill for source task $x$, and let $T_k(x)$ denote a task variant. The skill remains fixed when it is used on $T_k(x)$. We compare skill-assisted and no-skill executions of the same task or variant using its verifier. A positive paired difference indicates utility in that setting. Its persistence across changes indicates the observed scope of reuse.

We distinguish three questions. First, do positive source-task gains recur across runs? Second, how does the same skill perform when the representation, instance, or requirements change? Third, which guidance in the skill is associated with successful or failed reuse? The first two questions are answered by paired outcomes. The third is examined through skill contents, execution behavior, and the published construction procedures. The executor can adapt its actions on a variant, but the skill itself is not revised during this evaluation.

## 3. DIAGNOSING THE SCOPE OF SKILL UTILITY

We examine paired utility on standard tasks before evaluating frozen skills under three forms of task variation. The comparison establishes where the completed skills help and which changes alter their usefulness.

### 3.1. Experimental Setup

We evaluate SkillEvolver [1], SPARK [2], and CoEvoSkills [3] on SkillsBench [4], with two runs per method. Each run pairs the method's skill condition with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with $N$ tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left( r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

We also count tasks with positive, negative, and zero differences.

### 3.2. Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, CoEvoSkills gains 10.24 and 9.76, and SPARK gains 1.84 and 6.45. SkillEvolver and CoEvoSkills retain similar gains despite substantial changes in their no-skill scores. Their benefit is therefore more consistent across runs than their absolute skill-condition scores suggest.

The task-level results also differ. SkillEvolver has at least four times as many improvements as regressions in each run; CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK's first run is nearly balanced, while its second has 13 improvements and five regressions. These results establish recurring source-task utility. We next ask whether the same frozen skills remain useful after the task changes.

| Method | Run | $N$ | No skill | Skill | $\Delta$ | I/R/T |
|---|---|---|---|---|---|---|
| SkillEvolver | 1 | 81 | 30.20 | 55.81 | +25.61 | 27/6/48 |
| | 2 | 81 | 37.32 | 62.34 | +25.02 | 26/3/52 |
| SPARK | 1 | 84 | 17.82 | 19.66 | +1.84 | 9/8/67 |
| | 2 | 84 | 14.37 | 20.81 | +6.45 | 13/5/66 |
| CoEvoSkills | 1 | 85 | 42.21 | 52.44 | +10.24 | 17/9/59 |
| | 2 | 85 | 23.54 | 33.31 | +9.76 | 17/10/58 |

**Table 1**. Two runs per method on SkillsBench. Mean verifier rewards and paired gain $\Delta$ use a 0–100 scale. I/R/T counts improved, regressed, and tied tasks.

### 3.3. Task Changes Separate Different Scopes of Utility

We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) alter names, column arrangements, or schemas; task-instance changes (P2) replace the data or instance structure; coupled-requirement changes (P3) alter or add requirements that must be satisfied together. These variations test whether a frozen skill remains useful when the executor must reinterpret inputs, recompute a solution, or accommodate new requirements.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills, pairing each skill condition with the no-skill result on the same variant.

**Source-task improvement does not imply useful generalization.** All three methods improve mean reward on the source tasks in both runs, but their variant results diverge. SkillEvolver retains a positive mean gain of 11.94 points, SPARK is close to neutral at $+0.65$, and CoEvoSkills falls below the no-skill baseline at $-5.01$. SkillEvolver also improves more variants than it regresses (47 versus 20), whereas SPARK and CoEvoSkills show the reverse. Thus, the source-task gains of the latter two methods do not translate into a consistent advantage on changed tasks.

**Replacing the instance produces the largest divergence.** SkillEvolver remains positive under all three changes, with its largest gain on P2 ($+14.56$). P2 is also where SPARK and CoEvoSkills decline most ($-5.23$ and $-9.51$). SPARK retains positive averages under representation and requirement changes, but loses its advantage when the underlying instance changes. SkillEvolver's positive results across P1–P3 indicate a broader observed scope of skill utility. CoEvoSkills' negative averages across all three changes show that its source-task benefit is particularly sensitive to reuse conditions.

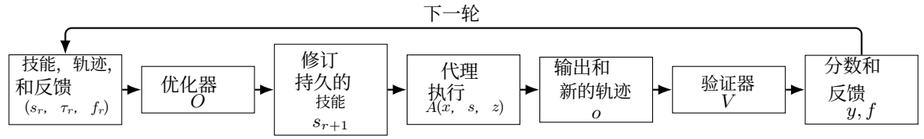



**图1**。技能进化和部署。我们的评估检验了完成技能的效用及其在任务变化下的重用；技能内容和执行轨迹为解释观察到的行为提供了证据。

### 2.2. 诊断技能学到了什么

令 $s_x$ 表示源任务 $x$的完成技能，并令 $T_k(x)$表示任务变体。当技能用于 $T_k(x)$时，它保持固定。我们使用其验证器比较相同任务或变体的技能辅助执行和无技能执行。正的配对差异表明在该环境中的效用。它在变化中的持久性表明观察到的重用范围。

我们区分了三个问题。首先，正面的源任务收益是否在多次运行中重复出现？其次，当表示、实例或要求发生变化时，同一技能的表现如何？第三，技能中的哪些指导与成功或失败的复用相关联？前两个问题由配对结果回答。第三个问题通过技能内容、执行行为以及已发布的构建程序来考察。执行者可以对其行为进行适应，但在此次评估过程中，技能本身并未被修订。

## 3. 诊断技能效用的范围

我们在评估冻结技能之前，先在标准任务上检查配对效用。比较结果确定了已完成技能的帮助范围，以及哪些变化会改变它们的实用性。

### 3.1. 实验设置

我们在 SkillsBench [4] 上评估 SkillEvolver [1]、SPARK [2] 和 CoEvoSkills [3]，每种方法运行两次。每次将技能条件与同题无技能执行配对。同一方法的两次运行使用相同任务集：SkillEvolver 81 题、SPARK 84 题、CoEvoSkills 85 题。

我们报告了0-100分制的平均验证者奖励，以及技能与非技能条件下的任务内平均差异。对于包含 $N$ 个任务的运行，配对增益为

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left( r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{4}$$

我们还统计了具有正差异、负差异和零差异的任务数量。

### 3.2. 源任务增益跨运行重现

三种方法在两次运行中的平均配对增益均为正（表1）。SkillEvolver 提高25.61和25.02分，CoEvoSkills 提高10.24和9.76分，SPARK 提高1.84和6.45分。前两者的无技能分数虽变化较大，增益却相近。因此，跨运行的收益比技能条件的绝对分数更稳定。

逐题结果也有差异。SkillEvolver 每次运行的改进数至少是退化数的四倍；CoEvoSkills 两次均改进17题，分别退化9题和10题。

SPARK 的第一次运行几乎平衡，而第二次运行中有 13 次改进和五次退化。这些结果确立了循环的源任务效用。我们接下来询问，任务变化后，相同的冻结技能是否仍然有用。

| Method | Run | $N$ | No skill | Skill | $\Delta$ | I/R/T |
|---|---|---|---|---|---|---|
| SkillEvolver | 1 | 81 | 30.20 | 55.81 | +25.61 | 27/6/48 |
| | 2 | 81 | 37.32 | 62.34 | +25.02 | 26/3/52 |
| SPARK | 1 | 84 | 17.82 | 19.66 | +1.84 | 9/8/67 |
| | 2 | 84 | 14.37 | 20.81 | +6.45 | 13/5/66 |
| CoEvoSkills | 1 | 85 | 42.21 | 52.44 | +10.24 | 17/9/59 |
| | 2 | 85 | 23.54 | 33.31 | +9.76 | 17/10/58 |

表1. SkillsBench 上每种方法的两次运行。平均验证器奖励及配对增益 Δ 均按0–100分制报告。I/R/T 分别为改进、退化和持平的任务数。

### 3.3. 任务变化分离不同效用范围

我们将每个源任务技能应用于受控变体，而无需进一步进化。输入表示变化（P1）会改变名称、列排列或模式；任务实例变化（P2）会替换数据或实例结构；耦合需求变化（P3）会改变或添加必须一起满足的需求。这些变化测试冻结的技能在执行者必须重新解释输入、重新计算解决方案或适应新需求时是否仍然有用。

完整记录包含 222 个变体和跨越无技能和三种方法的 886 个条件-等级结果。表 2 总结了 222 个 SkillEvolver 变体的历史和恢复评估，以及 SPARK 和 CoEvoSkills 各 221 个，将每个技能条件与同一变体上的无技能结果配对。

**源任务改进并不一定意味着有用的泛化**。所有三种方法在两次运行中均提高了源任务的平均奖励，但它们的变体结果出现了分歧。SkillEvolver 保持了 11.94 分的正面平均增益，SPARK 接近中性值 $+0.65$，而 CoEvoSkills 低于无技能基线值 $-5.01$。SkillEvolver 改进的变体比它退化的变体更多（47 对 20），而 SPARK 和 CoEvoSkills 则相反。因此，后两种方法的源任务增益并没有转化为在变化任务上的一致优势。

**替换实例会产生最大的差异**。 在所有三种变化下，SkillEvolver 均保持正值，其在 P2 上获得最大增益 ($+14.56$)。P2 也是 SPARK 和 CoEvoSkills 下降最严重的位置 ($-5.23$和 $-9.51$)。SPARK 在表示和需求变化下仍保持正值，但当底层实例变化时会失去优势。SkillEvolver 在 P1–P3 上的积极结果表明其观察到的技能效用范围更广。CoEvoSkills 在所有三种变化下的负平均值表明其源任务收益对重用条件特别敏感。

| A. Overall utility on task variants | | | | | |
|---|---|---|---|---|---|
| **Method** | $N$ | **No skill** | **Skill** | $\Delta$ | **I/R/T** |
| SkillEvolver | 222 | 41.87 | 53.81 | +11.94 | 47/20/155 |
| SPARK | 221 | 41.61 | 42.25 | +0.65 | 32/37/152 |
| CoEvoSkills | 221 | 41.61 | 36.60 | −5.01 | 26/41/154 |

| B. Utility by task change: $(N)\ \Delta$ | | | |
|---|---|---|---|
| **Method** | **P1** | **P2** | **P3** |
| SkillEvolver | (77) +11.85 | (68) +14.56 | (77) +9.72 |
| SPARK | (77) +2.00 | (68) −5.23 | (76) +4.53 |
| CoEvoSkills | (77) −3.59 | (68) −9.51 | (76) −2.42 |

**Table 2**. Skill utility under task variation: descriptive summary of historical and recovery evaluations. Each method is paired with its same-variant no-skill baseline. P1/P2/P3 denote representation, instance, and requirement changes. Panel B gives variant count $N$ and paired gain $\Delta$; scores and I/R/T follow Table 1.

## 4. EXPLAINING THE OBSERVED SCOPE OF REUSE

The variant results locate differences in skill utility. We examine two task families to make those differences interpretable: signal detection, where the executor must choose between plausible candidates, and scheduling, where the output must satisfy specific repair rules. These cases connect observed outcomes to the guidance preserved in the frozen skills.

### 4.1. Distinguishing a Candidate from a Valid Answer

In exoplanet detection, the SkillEvolver skill supports successful detection after input columns are rearranged (P1) and after the task requires two signals (P3). The CoEvoSkills skill fails both variants: its use leads to stellar modulation being reported as the target signal, and a signal and its harmonic being treated as two separate signals. Both skills fail the replacement-instance P2 variant.

The difference is visible in what the skills ask the executor to check. SkillEvolver separates candidate discovery, signal diagnosis, and refinement. Its guidance addresses misleading peaks, transit-like durations, and harmonic relationships. CoEvoSkills also includes validation, including phase-fold, odd/even, and residual checks, but its use in these variants does not resolve the misleading candidates. The presence of a checking routine is therefore less informative than whether its criteria distinguish the alternatives that matter on the changed task. SkillEvolver's successful P1 and P3 deployments illustrate that distinction, while its P2 failure shows that those criteria do not solve every new instance.

### 4.2. Preserving the Rules That Define a Correct Repair

In scheduling, the SkillEvolver and SPARK skills succeed on all three variants; the CoEvoSkills skill succeeds only on P3. In P1, using CoEvoSkills yields an otherwise feasible schedule that fails a required local-minimality check on the baseline-relative repair. Feasibility checks alone do not capture this local-minimality requirement.

The SkillEvolver skill explicitly retains local minimality and baseline-relative repair rules. These rules specify how an acceptable repair differs from an arbitrary feasible schedule, and remain relevant after the input names or scheduling instance change. The CoEvoSkills skill covers several structural and feasibility checks, yet the failed deployment misses the decisive repair condition. The contrast concerns which correctness conditions survive in the reusable guidance. SPARK's success across the same three variants also shows that this form of successful reuse is not exclusive to one method.

### 4.3. Construction Feedback and Skill Content

The construction procedures offer an interpretation of these skill-level differences. SkillEvolver compares successful and failed deployments of a candidate skill and patches missing or misleading guidance. Its independent audit also checks reliance on training-instance constants [1]. This exposes failures in using the skill and encourages rules that a later executor can apply to a different input. The scheduling and signal-detection skills reflect that emphasis by retaining criteria for checking a repair or discriminating between candidate signals.

SPARK grounds skill distillation in environment-verified trajectories, preserving execution evidence, checks, and lessons [2]. Its scheduling result shows that such evidence can support successful reuse. Its mixed aggregate profile raises a separate question about abstraction: a verified execution supplies evidence for what worked on the source task, while changed instances require determining which choices should be retained and which should be recomputed. The observed decline on P2 makes that distinction especially relevant to interpreting its results.

CoEvoSkills obtains detailed revision feedback from a co-evolving surrogate verifier, while the ground-truth oracle returns an opaque pass/fail signal [3]. This makes the surrogate's coverage of correctness conditions relevant to the guidance retained in the skill. This provides a plausible account of the scheduling failure: the skill preserves many checks, but successful reuse still depends on a condition those checks did not enforce in that deployment. Across the cases, feedback coverage and abstraction from the source instance are candidate explanations for the different scopes of utility.

## 5. IMPLICATIONS FOR REUSABLE SKILL LEARNING

### 5.1. Average Gain and Improvement Frequency

SPARK illustrates a distinction between the size and frequency of benefits. Its overall variant mean is slightly positive even though regressions outnumber improvements. A near-zero aggregate therefore does not mean that the skill has no effect: beneficial and harmful changes offset one another. SkillEvolver's positive mean is accompanied by more improvements than regressions, whereas CoEvoSkills has both a negative mean and more regressions. These outcome profiles describe whether a skill's aggregate utility reflects broad benefit among changed outcomes or a balance of opposing changes.

### 5.2. Implications for Skill Construction and Evaluation

The cases suggest a concrete target for skill construction: preserve how to recognize a valid result, as well as how to produce a candidate. A useful skill helps the executor decide when its procedure applies, what must be adapted, and which checks must hold before accepting the output. The feedback-based account connects this target to the methods' construction procedures as an interpretation of the observed patterns.

Evaluation should correspondingly report paired source-task utility, examine the same frozen artifact under distinct task changes,

| A. Overall utility on task variants | | | | | |
|---|---|---|---|---|---|
| **Method** | $N$ | **No skill** | **Skill** | $\Delta$ | **I/R/T** |
| SkillEvolver | 222 | 41.87 | 53.81 | +11.94 | 47/20/155 |
| SPARK | 221 | 41.61 | 42.25 | +0.65 | 32/37/152 |
| CoEvoSkills | 221 | 41.61 | 36.60 | −5.01 | 26/41/154 |

| B. Utility by task change: $(N)\ \Delta$ | | | |
|---|---|---|---|
| **Method** | **P1** | **P2** | **P3** |
| SkillEvolver | (77) +11.85 | (68) +14.56 | (77) +9.72 |
| SPARK | (77) +2.00 | (68) −5.23 | (76) +4.53 |
| CoEvoSkills | (77) −3.59 | (68) −9.51 | (76) −2.42 |

表2. 任务变化下的技能效用，汇总历史评估与补跑结果。每种方法均与同一变体的无技能基线配对。P1/P2/P3 分别为表示、实例和需求变化。B部分给出变体数 N 与配对增益 Δ；分数及 I/R/T 定义同表1。

## 4. 解释观察到的重用范围

变异结果定位了技能效用的差异。我们考察两个任务家族使这些差异可解释：信号检测，执行者必须在可信候选者之间选择，以及调度，输出必须满足特定的修复规则。这些案例将观察到的结果与冻结技能中保留的指导联系起来。

### 4.1. 区分候选答案与有效答案

在系外行星探测中，SkillEvolver 技能支持在输入列重新排列（P1）以及任务需要两个信号（P3）后成功探测。CoEvoSkills 技能在这两种情况下都失败：其使用会导致恒星调制被报告为目标信号，并且一个信号及其谐波被视为两个独立的信号。这两种技能都未能通过替换实例 P2 变体。

这种差异体现在技能要求执行者检查的内容上。SkillEvolver 将候选发现、信号诊断和精炼分开。其指导内容涉及误导性峰值、类凌日持续时间和谐波关系。CoEvoSkills 也包括验证，包括相位折叠、奇偶和残差检查，但在这些变体中使用时，它并未解决误导性候选问题。因此，检查程序的存不存在不如其标准能否区分变化任务中重要的替代方案那么重要。SkillEvolver 的 P1 和 P3 成功部署说明了这一点，而其 P2 失败则表明这些标准并非能解决所有新实例。

### 4.2. 保留定义正确修复的规则

在调度中，SkillEvolver 和 SPARK 技能都在三种变体上成功；CoEvoSkills 技能仅在 P3 上成功。在 P1 中，使用 CoEvoSkills 产生一个原本可行的调度，但在基线相对修复上未能通过所需的局部最小性检查。仅进行可行性检查并不能捕获这一局部最小性要求。

SkillEvolver技能明确保留了局部最小性和基线相对的修复规则。这些规则指定了可接受的修复与任意可行的调度有何不同，并且在输入名称或调度实例发生变化后仍然相关。Co-EvoSkills技能涵盖了多个结构和可行性检查，然而失败的部署错过了决定性的修复条件。对比在于哪些正确性条件在可重用指导中得以保留。SPARK在相同的三个变体中取得的成功也表明，这种形式的成功重用并非仅限于一种方法。

### 4.3. 构造反馈与技能内容

构造程序为这些技能水平差异提供了一种解释。SkillEvolver 比较候选技能的成功和失败部署，并修补缺失或误导性的指导。其独立的审计还会检查对训练实例常量 [1] 的依赖。这暴露了使用技能的失败，并鼓励后来执行者可以应用于不同输入的规则。调度和信号检测技能通过保留检查修复或区分候选信号的标准，反映了这种侧重。

SPARK 从经环境验证的轨迹中蒸馏技能，保留执行证据、检查与经验教训 [2]。其调度结果说明，这类证据可以支持成功复用。总体结果有升有降，进一步引出了抽象问题：已验证的执行说明哪些做法在源任务上有效；实例变化后，仍需判断哪些选择应保留、哪些应重新计算。P2 上的下降使这一区别对解释其结果尤为重要。

CoEvoSkills 从协同进化的代理验证器处获取详细的修订反馈，而真实值预言机则返回一个不透明的通过/失败信号 [3]。这使得代理对正确性条件的覆盖与技能中保留的指导相关。这为调度失败提供了一个合理的解释：技能保留了许多检查，但成功的重用仍然取决于那些检查在那个部署中未强制执行的条件。在所有案例中，反馈覆盖率和从源实例的抽象是效用不同范围候选的解释。

## 5. 可重用技能学习的启示

### 5.1. 平均增益和改进频率

SPARK 说明了收益的大小和频率之间的区别。尽管退化多于改进，但其整体变体均值略为正。因此，接近零的总量并不意味着技能没有效果：有益和有害的变化相互抵消。SkillEvolver 的正均值伴随着比退化更多的改进，而 CoEvoSkills 则具有负均值和更多的退化。这些结果配置描述了技能的总体效用是否反映了变更结果中的广泛益处，还是相反变化的平衡。

### 5.2. 技能构建与评估的意义

这些案例为技能构建提供了一个具体的目标：保留识别有效结果的方法，以及如何生成候选方案。一个有用的技能能帮助执行者决定其程序何时适用、哪些需要调整，以及在接受输出前必须满足哪些检查。基于反馈的说明将这一目标与方法的构建过程联系起来，将其解释为对观察到的模式的解读。

因此，评估应报告源任务的配对效用，在不同任务变化下检验同一冻结技能，

and inspect cases where the outcomes diverge. The three probes describe a profile of applicability: passing one change does not substitute for checking another. Reporting that profile alongside improvement and regression counts exposes behavior that a single average hides. It also makes the learning question concrete: which decisions remain useful when the source task no longer supplies the same inputs or demands the same output?

## 6. RELATED WORK

**Learning persistent skills from experience.** SkillEvolver, SPARK, and CoEvoSkills make experience persistent through different construction loops: deployment-grounded revision, verified-trajectory distillation, and co-evolutionary verification, respectively [1–3]. These methods motivate our focus on the artifact delivered to a later executor. We study how that artifact's usefulness changes after its source task is varied, and interpret the resulting behavior through its retained guidance.

**Evaluating skill utility and use.** SkillsBench evaluates skill utility across diverse tasks, while SWE-Skills-Bench studies skill use in software engineering [4, 5]. SkillLearnBench evaluates continual skill generation on real-world tasks [6]. From Raw Experience to Skill Consumption separates experience generation, skill extraction, and downstream consumption [7]. Our study complements these evaluations by connecting repeated source-task outcomes with controlled variations of the task and skill-level examples. The central object is the scope of usefulness of a frozen skill, rather than its score on the source task alone.

## 7. CONCLUSION

We examine what self-evolving agent skills preserve through repeated standard-task evaluations, task variation, and inspection of skill reuse. SkillEvolver, SPARK, and CoEvoSkills all yield positive source-task gains in both runs, but their variant results differ substantially. SkillEvolver has the broadest observed generalization profile, remaining positive under representation, instance, and requirement changes. SPARK's benefit depends on the change type, while CoEvoSkills' averages become negative. The inspected examples associate successful reuse with guidance for selecting valid results and preserving task-specific correctness rules. Together, these findings show why diagnosing skill learning requires examining how a frozen artifact guides behavior after the task changes.

并检查结果出现分歧的案例。这三个探针描述了适用性特征：通过一个变化并不能替代检查另一个。在改进和回归计数旁边报告该特征揭露了单一平均值隐藏的行为。它还使学习问题具体化：当源任务不再提供相同输入或要求相同输出时，哪些决策仍然有用？

## 6. 相关工作

**从经验中学习持久的技能**。 SkillEvolver、SPARK 和 CoEvoSkills 通过不同的构建循环使经验持久化：基于部署的修订、验证轨迹的蒸馏和协同进化的验证，分别 [1–3]。这些方法促使我们关注传递给后续执行者的工件。我们研究该工件在源任务变化后的有用性如何变化，并通过其保留的指导来解释其行为。

评估技能效用与使用。SkillsBench 覆盖多样任务，SWE-Skills-Bench 研究软件工程中的技能使用 [4,5]。SkillLearnBench 评估真实任务上的持续技能生成 [6]。《从原始经验到技能使用》区分经验生成、技能提取与下游使用 [7]。我们的研究将重复的源任务结果、受控任务变化和技能层面的案例联系起来，补充这些评估。核心研究对象是冻结技能的适用范围，而不仅是它在源任务上的分数。

## 7. 结论

我们考察了自进化代理技能在重复标准任务评估、任务变化和技能重用检查中保留的内容。SkillEvolver、SPARK和CoEvoSkills在两次运行中都产生了正的源任务增益，但它们的变体结果差异很大。SkillEvolver具有最广泛的观察到的泛化特征，在表示、实例和需求变化下仍保持正值。SPARK的收益取决于变化类型，而CoEvoSkills的平均值变为负数。检查的示例将成功的重用与选择有效结果的指导以及保留特定任务正确性规则相关联。这些发现共同表明，为什么诊断技能学习需要检查在任务变化后，冻结的工件如何指导行为。

## 8. REFERENCES


[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.



## 8. REFERENCES

[1] Genrui Zhang, Erle Zhu, Jinfeng Zhou, Caiyan Jia, and Hongning Wang, "SkillEvolver: Skill Learning as a Meta-Skill," 2026.

[2] Yang Zhou, Zihan Dong, Zhenting Wang, Can Jin, Shiyu Zhao, Bangwei Guo, Difei Gu, Linjun Zhang, Mu Zhou, and Dimitris N. Metaxas, "Evidence Over Plans: Online Trajectory Verification for Skill Distillation," 2026.

[3] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, Xue Liu, Xiaoxiao Li, and Philip S. Yu, "CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification," 2026.

[4] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song, "SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks," 2026.

[5] Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu, "SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering?," 2026.

[6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong, "SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks," 2026.

[7] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo, "From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills," 2026.